Shahbaz Singh 0263901s

3rd Check-in

I'm a bit behind on this one. My research question is still the same as in check-in 1, but since check-in 2 I've mostly kept working through the basics, going through Andrej Karpathy's video lessons on how language models are built, because I wanted to understand how the model works before running experiments on it. So I don't have any results yet, nothing has been trained at full size, and the methods section in my paper is still a rough first draft.

As a reminder, my project uses nanoGPT, a small open-source version of the kind of model behind ChatGPT. It reads Shakespeare one character at a time and learns to guess the next character. Keller Jordan's modded-nanoGPT speedrun is a project where people try to train this kind of model as fast as possible, and it stacks a lot of tricks together, so it's hard to tell which ones really help. I want to test three of them (RoPE, RMSNorm and Muon) one at a time against the plain model. The main thing I'm comparing is validation loss, which is a score for how well the model guesses text it didn't see during training.

The main idea I've been learning is attention, which is how the model decides what earlier text matters. To guess the next character, it looks back at the earlier ones and gives more weight to the ones that seem useful. One thing I learned is that attention doesn't know the order of the characters on its own, so position has to be given to the model separately.

This helped me see that the three tricks change different parts of the model. RoPE changes how the model keeps track of where each character is. RMSNorm changes normalization, a step that keeps the numbers inside the model in a stable range so training doesn't go wrong. Muon changes how the model learns from its mistakes. It's a different optimizer, which is the method that adjusts the model's weights during training. I get what Muon does but I'm still not sure why it trains better.

My starting point is nanoGPT's standard Shakespeare setup, which has about 10.65 million weights and trains for 5,000 steps. Its author gets a validation loss of around 1.47, so next I want to reproduce that on Colab. Then I'll train each trick with at least 3 different seeds and use a t-test to compare them like I said in check-in 2.

In check-in 2 I also listed a few things I wasn't sure about. For Muon's learning rate, I'm now thinking of giving every version the same small learning rate search, so that none of them gets extra tuning. For matching compute, I'm planning to report Muon both at the same number of steps and at the same training time, since each Muon step takes longer. I'm still not sure if 3 seeds is enough to see small differences, so I might go up to 5. If there's time and budget left after that, I'd still like to try the tricks that helped on a small GPT-2 model. Since I'm behind, my results for check-in 4 might be partial.